

MLP

讲解MLP

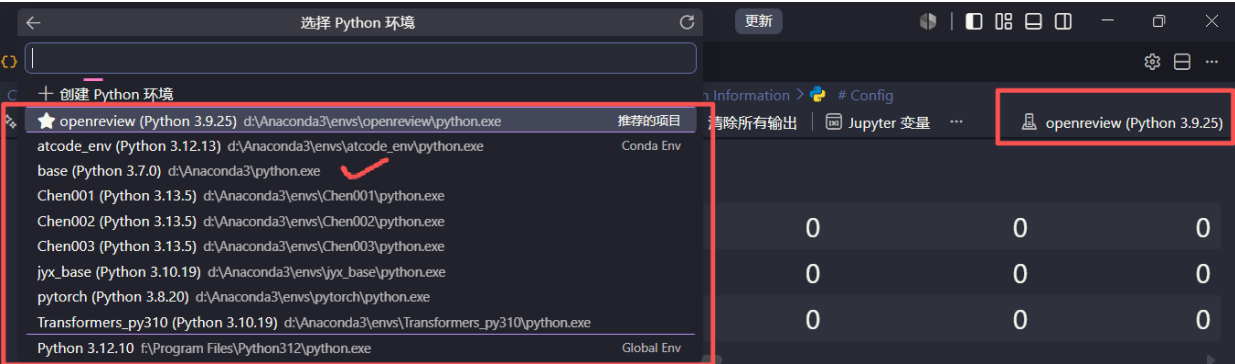
https://www.bilibili.com/video/BV1JE411g7XF?vd_source=d4b168555dc577426edfdabdc07967&p=12&spm_id_from=333.788.videopod.episodes

环境

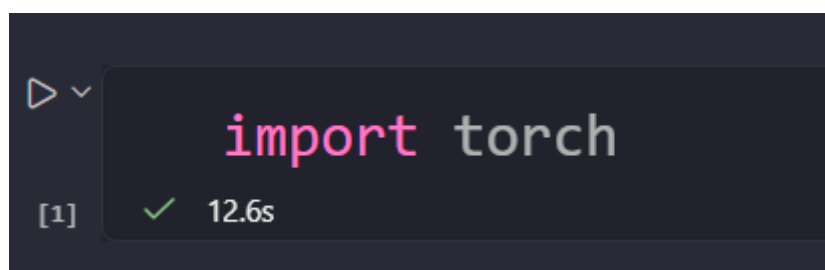
从mlp开始，咱们就正式进入深度学习的领域了，这是我之前写的安装教程，<https://www.cnblogs.com/DLChen/p/18171693>，可以参考安装一下

合集 - 深度学习(8)	
1. 下载和安装anaconda	2024-05-03
2. pytorch安装教程	2024-05-04
3. 安装Sublime Text3+pytorch虚拟环境	2024-05-17
4. VScode和python解释器	2024-05-04
5. 全连接神经网络	2024-12-09
6. 卷积神经网络	2024-12-11
收起	

也可以去b站搜索相关的教程



只要存在除了base环境之外的，有其他的就行，然后选择对应的环境之后，运行下面的代码，不报错，就ok了



MLP基础代码

```
#创建一个 cell
```

#%%

```
import torch
```

```
import torchvision
```

```
import torchvision.transforms as transforms
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import matplotlib.pyplot as plt
```

```
from torchvision import datasets
```

数据集准备

```
transform = transforms.Compose([
```

```
transforms.Grayscale(num_output_channels=1), # 转换为单通道灰度图
```

```
transforms.ToTensor() # 转换为张量
```

D

训练集

```
train_dataset = datasets.ImageFolder(root='./mnist_images/train', transform=transform)
```

```
#train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64,
```

```
# shuffle=True, num_workers=2)
```

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64,
```

```
shuffle=True)
```

测试集

```
test_dataset = datasets.ImageFolder(root='./mnist_images/test', transform=transform)
```

```
#test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000,
```

```
# shuffle=True, num_workers=2)
```

```
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000,
```

```
shuffle=True)
```

```
#%%
```

```
# 模型搭建阶段
```

```
class LinearModel(nn.Module):
```

```
    def __init__(self):
```

```
        super(LinearModel, self).__init__()
```

```
        self.fc1 = nn.Linear(28 * 28, 256) # 输入大小为 28*28, 输出大小为 10
```

```
(0-9 十个数字)
```

```
        self.fc2 = nn.Linear(256, 10)
```

```
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
```

```
        x = x.view(-1, 28 * 28) # 将图片展开成一维向量
```

```
        x = self.fc2(self.relu(self.fc1(x))) # 全连接层
```

```
        return x
```

```

model = LinearModel()

# 损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# 训练和测试阶段
def train_model(model, train_loader, criterion, optimizer, num_epochs=5):
    model.train()
    for epoch in range(num_epochs):
        running_loss = 0.0
        for i, data in enumerate(train_loader, 0):
            inputs, labels = data
            optimizer.zero_grad() # 1.将梯度清零
            outputs = model(inputs) # 2.计算神经网络的前向传播结果
            loss = criterion(outputs, labels) # 3.计算 output 和标签 label 之间的损失
            loss.backward() # 4.使用 backward 计算梯度
            optimizer.step() # 5.使用 optimizer.step 更新参数

            running_loss += loss.item()
            if i % 100 == 99: # 每 100 个 batch 打印一次损失值
                print(f'Epoch [{epoch + 1}/{num_epochs}], Step [{i + 1}/{len(train_loader)}], Loss: {running_loss / 100:.4f}')
                running_loss = 0.0

def test_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data in test_loader:
            images, labels = data
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    print(f'Test Accuracy: {accuracy:.2f}%')

# 训练模型
train_model(model, train_loader, criterion, optimizer)

```

```

# 测试模型
test_model(model, test_loader)

###
#展示错误预测的样本
def get_wrong_predictions(model, test_loader):
    wrong_images = []
    wrong_labels = []
    wrong_preds = []
    correct = 0 # 记录正确预测的数量
    total = 0 # 记录总样本数量

    with torch.no_grad():
        for data in test_loader:
            images, labels = data
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            # 统计总样本数
            total += labels.size(0)
            # 统计正确预测的数量
            correct += (predicted == labels).sum().item()

            # 筛选预测错误的样本
            wrong_idx = (predicted != labels).nonzero(as_tuple=True)[0]
            wrong_images.append(images[wrong_idx])
            wrong_labels.append(labels[wrong_idx])
            wrong_preds.append(predicted[wrong_idx])

    # 拼接所有错误预测的样本
    wrong_images = torch.cat(wrong_images)
    wrong_labels = torch.cat(wrong_labels)
    wrong_preds = torch.cat(wrong_preds)

    # 计算准确率
    accuracy = 100 * correct / total
    print(f'Test Accuracy: {accuracy:.2f}%')

    return wrong_images, wrong_labels, wrong_preds

# 展示预测错误的图片和预测结果
def show_wrong_predictions(wrong_images, wrong_labels, wrong_preds,

```

```
num_images=6):
    np_images = wrong_images.numpy()

    fig, axes = plt.subplots(1, num_images, figsize=(12, 3))

    for i in range(num_images):
        ax = axes[i]
        img = np_images[i].squeeze()
        ax.imshow(img, cmap='gray')
        ax.set_title(f'True: {wrong_labels[i].item()} Pred: {wrong_preds[i].item()} ',
loc='left')
        ax.axis('off')

    plt.show()

# 获取预测错误的图片和对应的标签
wrong_images, wrong_labels, wrong_preds = get_wrong_predictions(model,
test_loader)

# 展示 6 张预测错误的图片
show_wrong_predictions(wrong_images, wrong_labels, wrong_preds, num_images=6)
```

拓展任务

全连接神经网络的扩展实验任务

- (1) 针对手写数字集，建立全连接神经网络，至少包含 2 个隐藏层。
- (2) 将每个类别的图片（在 `train` 文件夹中的）随机打乱并等分为 5 份，然后将前 4 份用于训练，第 5 份用于测试。
- (3) 列出 1 个隐藏层神经网络和 2 个、3 个隐藏层神经网络的预测准确率的比较结果。
- (4) 列出具有两个 `hidden layer`，使用不同神经元个数（例如第 1 层神经元个数为 100，第 2 层神经元个数为 50）的比较结果。

1.根据MLP的基础代码完成MLP的拓展任务

2.完成拓展任务后进行调参，比如：epoch训练轮数、MLP层数、隐藏层神经元个数、优化器、学习率、Dropout、权重衰减等。各参数的用法以及作用自行查找资料。参数调整的建议：每次调整一次参数在文档中记录参数的变化和对应的模型性能。每次最好只调整一个参数的变化、找到好的参数在此基础上做进一步调整，最后将所有参数整理好可以与我进行讨论。

先拿手写数字集的数据来做MLP的练习

参考

关于参数调整，给一个参考，这是之前咱们组本科生完成的

参数变化	acc	增长率
5轮, 2层, 256, 64, SGD优化器, lr=0.01, dropout=0	91.80%	/
5轮, 2层, 256, 64, ADam优化器, lr=0.01, dropout=0	96.59%	5.22%
5轮, 2层, 256, 64, ADamW优化器, lr=0.01, dropout=0	96.94%	5.60%
15轮, 2层, 256, 64, ADamW优化器, lr=0.01, dropout=0	97.13%	5.81%
15轮, 2层, 256, 128, ADamW优化器, lr=0.01, dropout=0	97.35%	6.05%
15轮, 2层, 128, 64, ADamW优化器, lr=0.01, dropout=0	96.39%	4.99%
15轮, 2层, 512, 256, ADamW优化器, lr=0.01, dropout=0	96.94%	5.60%
15轮, 2层, 512, 128, ADamW优化器, lr=0.01, dropout=0	97.32%	6.01%
15轮, 2层, 512, 64, ADamW优化器, lr=0.01, dropout=0	96.98%	5.64%
15轮, 2层, 256, 128, ADamW优化器, lr=0.01, dropout=0.1	97.57%	6.29%
15轮, 2层, 256, 128, ADamW优化器, lr=0.01, dropout=0.3	97.38%	5.08%
15轮, 2层, 256, 128, ADamW优化器, lr=0.01, dropout=0.5	96.85%	5.50%
15轮, 2层, 256, 128, ADamW优化器, lr=0.001, dropout=0.1	98.16%	6.93%
15轮, 2层, 256, 128, ADamW优化器, lr=0.005, dropout=0.1	97.47%	6.18%
15轮, 2层, 256, 128, ADamW优化器, lr=0.001, dropout=0.1, 权重衰减	98.13%	6.90%